Epic: AI News Assistant

Nov 13, 2026 · @Krati Goyal

Summary

Build a multi-agent assistant that turns a technology topic into a short, audience-specific news briefing with sources and a confidence label on every point. The core pipeline is working (18 of 44 story points done); the remaining work makes the briefings cleaner, smarter and deliverable.

Field	Value
Epic key	NA-E1
Title	AI News Assistant: audience-aware tech news briefings
Priority	High
Stories	12 (5 done, 7 to do)
Estimate	44 story points
Components	Research Agent, Analysis Agent, Notifier Agent, LangGraph workflow

Problem and goals

Teams lose hours each week scanning scattered tech news, and the same story matters differently to an engineer, a security analyst and a manager. Automated summaries also hide how reliable they are.

This epic delivers four outcomes:

1. **Save reading time:** one briefing of the 5 most important points per topic, from news published in the past 7 days.
2. **Fit the reader:** four audience versions (Engineering, Cybersecurity, Management, All Employees), each with its own intro and recommended action.
3. **Stay transparent:** every point shows its source links, priority, confidence and verification status, plus a disclaimer.

4. **Stay modular:** three independent agents sharing one state, so any step can be upgraded (for example, rule-based analysis to an LLM) without touching the others.

Scope

In scope

- Command-line input of a topic and one of four audiences
- News search through the Tavily API (news mode, past week, up to 10 articles)
- Key-point extraction, deduplication, scoring and corroboration
- Audience-specific briefing saved as a timestamped text file in `outputs/`
- LangGraph workflow that stops early on errors or empty results
- Text cleaning, accurate keyword matching, a research retry loop and optional LLM summaries
- Delivery by email or Slack, a README and automated tests

Out of scope

- A graphical or web interface (candidate for a follow-up epic)
- Non-English news and translation
- Independent fact-checking beyond source-to-source comparison
- User accounts, saved preferences or a briefing history

User stories

Twelve stories across six features. The five marked Done are in the current code; the rest come from the issues found in the test runs.

ID	Feature	User story	Points	Status
NA-1	Input	As a user, I want to enter a topic and pick my audience, so the briefing fits my team.	2	Done
NA-2	Research	As a user, I want recent news on my topic found automatically, so I don't search site by site.	5	Done
NA-3	Research	As a user, I want the assistant to search again when it finds too few articles, so thin topics still get a briefing.	3	To do

ID	Feature	User story	Points	Status
NA-4	Analysis	As a reader, I want the 5 most important points picked out, so I can catch up in minutes.	5	Done
NA-5	Analysis	As a reader, I want web-page clutter removed from key points, so each point reads as a clean sentence.	3	To do
NA-6	Analysis	As a reader, I want priority labels I can trust, so "high" actually means important.	2	To do
NA-7	Analysis	As a reader, I want points summarised and cross-checked by an LLM, so confidence reflects real agreement between sources.	8	To do
NA-8	Notification	As a team lead, I want a briefing written for my audience and saved to a file, so I can share it.	3	Done
NA-9	Notification	As a team lead, I want the briefing sent by email or Slack, so my team gets it without extra steps.	5	To do
NA-10	Workflow	As a developer, I want the agents wired in a LangGraph workflow that stops on errors, so no empty report is produced.	3	Done
NA-11	Quality	As a developer, I want a README and safe handling of API keys, so others can run the project securely.	2	To do
NA-12	Quality	As a developer, I want automated tests for each agent and the workflow, so changes don't break the pipeline.	3	To do

Acceptance criteria

1. **NA-1 Input**
   - An empty topic is rejected with a message.
   - Choices 1–4 map to Engineering, Cybersecurity, Management and All Employees.
   - Any other choice defaults to All Employees.
2. **NA-2 Research**
   - Searches Tavily in news mode for the past week, up to 10 results.

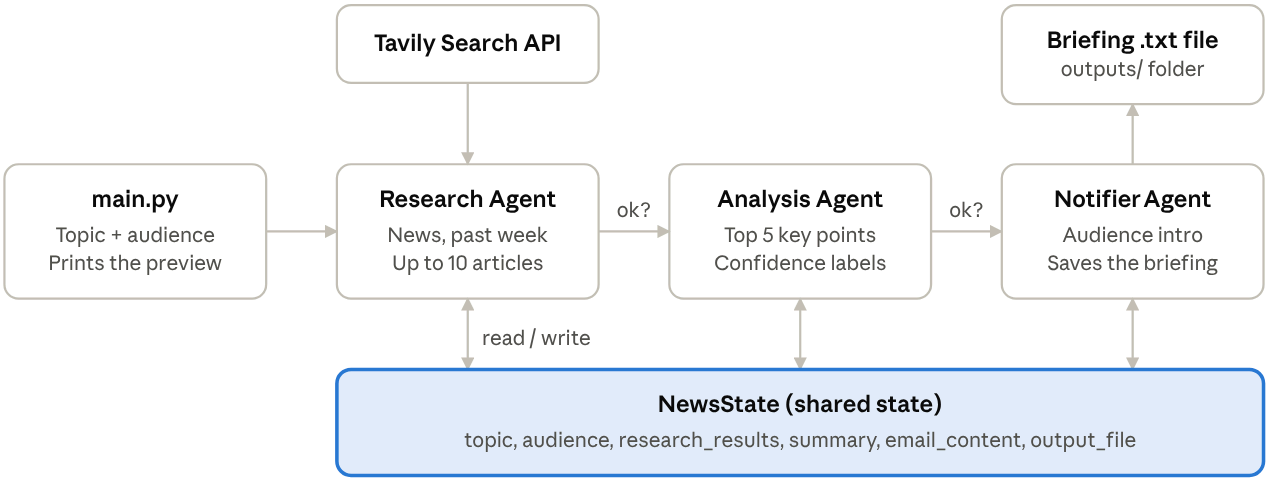
- Results with a missing or repeated URL are skipped.
- Each article keeps title, description, source, published date and URL.
- A missing `TAVILY_API_KEY` or a search error returns a clear message and the workflow stops.

3. **NA-3 Research retry**
   - If fewer than 3 articles return, research runs again with a broader query.
   - `research_attempts` increases each time; at most 3 attempts in total.
   - After the last attempt, the workflow continues with what it has or stops with a message.
4. **NA-4 Key points**
   - Only sentences of 8 or more words are considered.
   - Sentences more than 90% similar are kept once.
   - Each of the top 5 points carries status, confidence, priority and source URLs.
5. **NA-5 Clean text**
   - Markdown headings, "[...]", navigation text ("Sign in", "Read more") and bylines are stripped.
   - Re-running the 3 test topics gives 0 points with clutter (baseline: 13 of 20).
6. **NA-6 Priority**
   - Keywords match whole words only, so "said" and "chair" no longer count as "ai".
   - In the test set, not every point is rated high (baseline: 20 of 20).
7. **NA-7 LLM analysis**
   - Points are summarised through `langchain-openai` when `OPENAI_API_KEY` is set.
   - Corroboration uses semantic similarity instead of character matching.
   - Without a key, the rule-based analysis runs as today.
8. **NA-8 Briefing**
   - Intro and recommended action match the chosen audience.
   - Briefing lists the top 5 points, verification notes and a disclaimer.
   - File is saved as `outputs/news_report_<topic>_<timestamp>.txt` with unsafe characters replaced.
9. **NA-9 Delivery**
   - Briefing can be sent to a configured email address or Slack channel.
   - Credentials come from `.env`; a failed send keeps the saved file and shows an error.

10. **NA-10 Workflow**
    - Flow runs research, then analysis, then notifier.
    - It ends early when there is an error, no articles or no summary.
    - `main.py` prints topic, audience, counts, file path and a preview.
11. **NA-11 Docs and keys**
    - README covers setup, `.env` keys and how to run.
    - `.env` and `.venv` are never shared in archives; exposed keys are rotated.
12. **NA-12 Tests**
    - Unit tests cover each agent with the Tavily client mocked.
    - Workflow tests cover the continue and stop routes.

Technical design

Three agents pass one shared state from search to saved briefing



Each ok? check ends the run early when there is an error or nothing to pass on.

LangGraph workflow · 3 agents, 2 checks, 1 shared state

The agents never call each other: LangGraph runs them in order and each one reads from and writes to `NewsState`. That is what lets NA-7 swap in an LLM without touching research or notification.

Stack: Python, LangGraph, Tavily (`tavily-python`), `python-dotenv`, `difflib`; `langchain-openai` for NA-7.

Definition of done and success metrics

The epic is done when all 12 stories meet their acceptance criteria and the targets below hold on the three regression topics ("AI", "Nvidia CEO", "Super Mario").

Metric	Baseline (13 Nov 2026 runs)	Target
Key points containing web-page clutter	13 of 20	0
Points rated "high" priority	20 of 20	Under 60%
Points corroborated by a second source	0 of 20	At least 30%
Duplicate points within one briefing	1	0
Runs that crash instead of showing an error	0	0

Each story is also done only when its code is reviewed, its tests pass, and the README reflects any new setup step.

Risks, dependencies and assumptions

The biggest risk is a misleading briefing, so every point keeps its sources and the disclaimer stays in place.

Risk	Impact	Mitigation
Scraped text is noisy or off-topic	Unclear or irrelevant key points	NA-5 cleaning; topic-relevance check in NA-7
Readers treat points as verified facts	Wrong decisions based on a report	Confidence and status labels, source links, disclaimer
API keys leak (they were found inside a shared zip)	Unauthorised use and charges	Rotate keys; keep `.env` out of archives (NA-11)
Tavily or OpenAI limits, outages or price changes	Failed or costly runs	Clear error messages; rule-based fallback when no LLM key
LLM summaries add errors	Points that no source supports	Keep source URLs per point; compare summary to source text

Dependencies: Tavily API key; OpenAI API key for NA-7 only; email or Slack credentials for NA-9; Python 3 with LangGraph and `tavily-python`.

Assumptions: briefings are in English; news from the past 7 days is recent enough; one user runs the tool at a time from the command line.